

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут телекомунікацій, радіоелектроніки та електронної техніки
кафедра «Радіоелектронні пристрої та системи»



Звіт з лабораторної роботи №1
з дисципліни «Об'єктно-орієнтоване програмування, частина 2»

Підготував:
ст. групи ІХ-21
Щурко Максим Олександрович

Прийняла:
Гордійчук-Бублівська О.В

Львів 2025р.

Тема роботи: Застосування принципів SOLID об'єктно-орієнтованого програмування при проєктуванні програмних систем у сфері телекомунікацій

Мета роботи: Ознайомитися з принципами SOLID. Навчитися застосовувати їх під час проєктування класів на Python. Формувати навички створення гнучкого, масштабованого та підтримуваного коду.

Теоретичні відомості

SOLID — це набір із п'яти принципів об'єктно-орієнтованого програмування, запропонованих Робертом Мартіном, які спрямовані на покращення якості архітектури програмного забезпечення.

S — Single Responsibility Principle (SRP)

Кожен клас повинен мати лише одну відповідальність і, відповідно, одну причину для зміни.

O — Open/Closed Principle (OCP)

Програмні сутності повинні бути відкритими для розширення, але закритими для модифікації.

L — Liskov Substitution Principle (LSP)

Об'єкти підкласів повинні повністю замінювати об'єкти базового класу без порушення логіки програми.

I — Interface Segregation Principle (ISP)

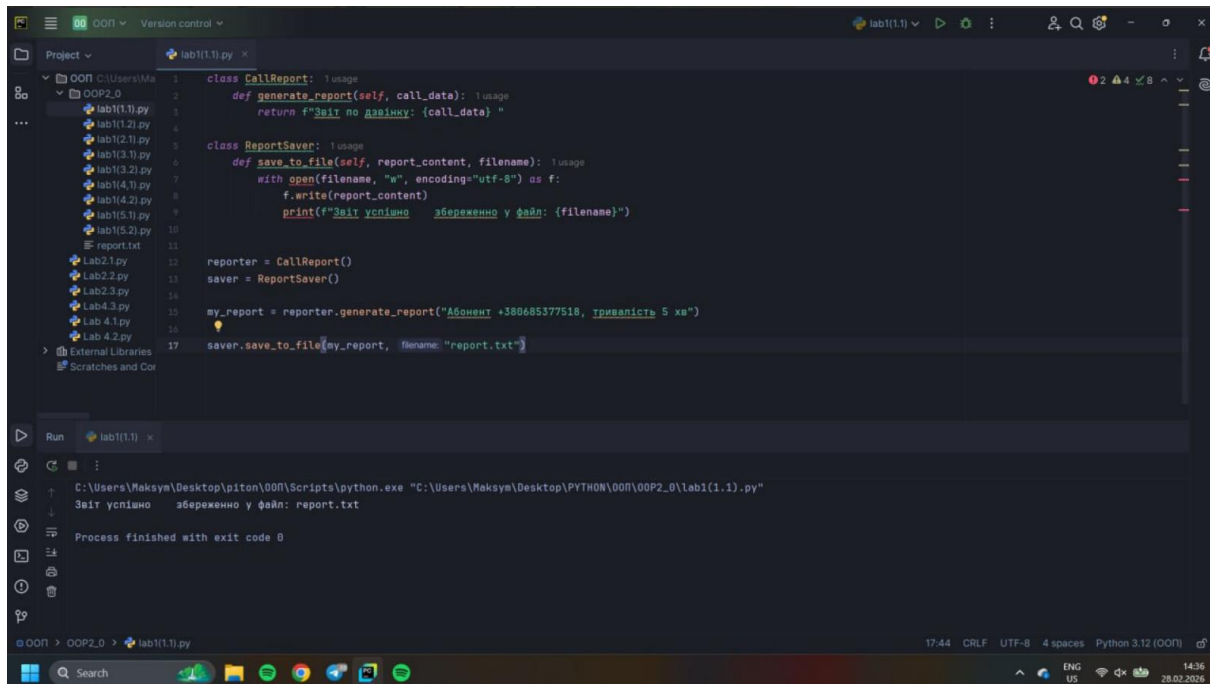
Краще мати кілька спеціалізованих інтерфейсів, ніж один універсальний.

D — Dependency Inversion Principle (DIP)

Модулі верхнього рівня не повинні залежати від модулів нижнього рівня.

Обидва повинні залежати від абстракцій.

Завдання



```
1 class CallReport:
2     def generate_report(self, call_data):
3         return f"Звіт по дзвінку: {call_data}"
4
5 class ReportSaver:
6     def save_to_file(self, report_content, filename):
7         with open(filename, "w", encoding="utf-8") as f:
8             f.write(report_content)
9         print(f"Звіт успішно збережено у файл: {filename}")
10
11 reporter = CallReport()
12 saver = ReportSaver()
13
14 my_report = reporter.generate_report("Абонент +380685377518, тривалість 5 хв")
15
16 saver.save_to_file(my_report, filename="report.txt")
```

Run lab1(1.1) x

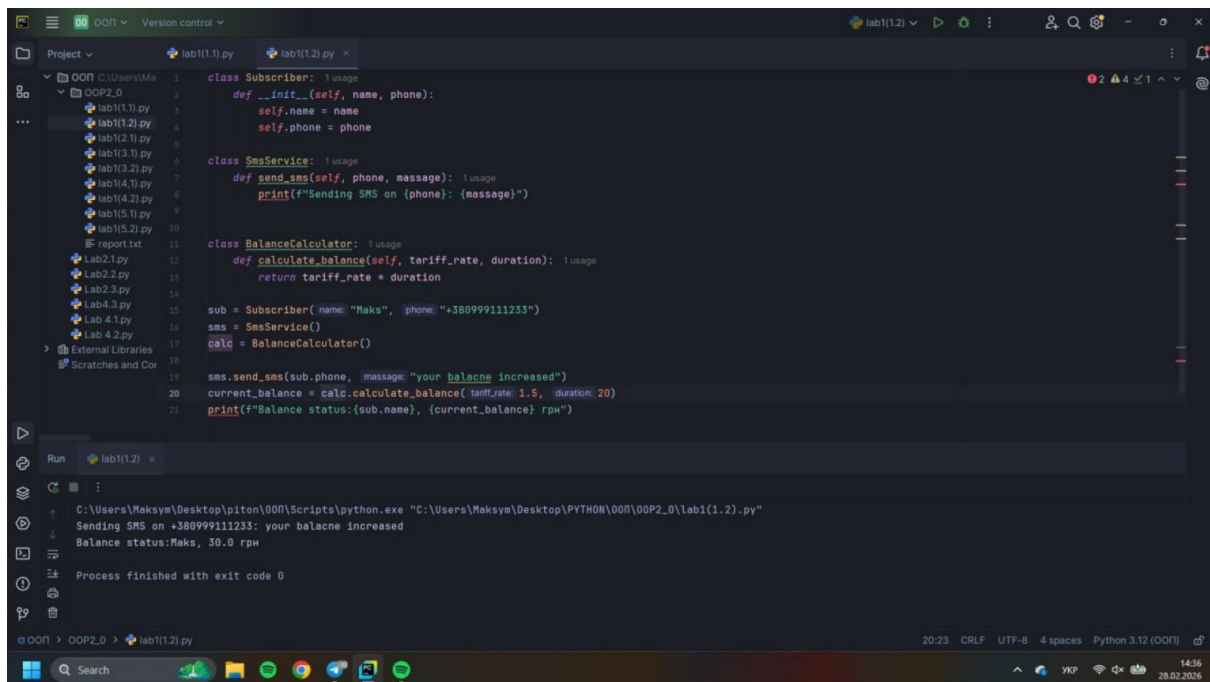
C:\Users\Maksym\Desktop\python\000\Scripts\python.exe "C:\Users\Maksym\Desktop\PYTHON\000\00P2_0\lab1(1.1).py"

Звіт успішно збережено у файл: report.txt

Process finished with exit code 0

17:44 CRLF UTF-8 4 spaces Python 3.12 (OOPI) ENG US 1436 28.02.2024

Рис. 1(Завдання 1.1)



```
1 class Subscriber:
2     def __init__(self, name, phone):
3         self.name = name
4         self.phone = phone
5
6 class SmsService:
7     def send_sms(self, phone, message):
8         print(f"Sending SMS on {phone}: {message}")
9
10 class BalanceCalculator:
11     def calculate_balance(self, tariff_rate, duration):
12         return tariff_rate * duration
13
14 sub = Subscriber(name="Maks", phone="+380999111233")
15 sms = SmsService()
16 calc = BalanceCalculator()
17
18 sms.send_sms(sub.phone, message="your balance increased")
19 current_balance = calc.calculate_balance(tariff_rate=1.5, duration=20)
20 print(f"Balance status:{sub.name}, {current_balance} грн")
```

Run lab1(1.2) x

C:\Users\Maksym\Desktop\python\000\Scripts\python.exe "C:\Users\Maksym\Desktop\PYTHON\000\00P2_0\lab1(1.2).py"

Sending SMS on +380999111233: your balance increased

Balance status:Maks, 30.0 грн

Process finished with exit code 0

20:23 CRLF UTF-8 4 spaces Python 3.12 (OOPI) UKR 1436 28.02.2024

Рис.2(Завдання 1.2)

```
1 #2.1
2 class ServiceTariff:
3     def calculate(self, units):
4         return 0
5
6 class VoiceTariff(ServiceTariff):
7     def calculate(self, minutes):
8         return minutes * 0.75 # 0.75 грн за хвилину розмови
9
10 class DataTariff(ServiceTariff):
11     def calculate(self, megabytes):
12         return megabytes * 0.15 # 0.15 грн за МБ даних
13
14 # 2.2: Розширення системи новим класом RoamingTariff
15 class RoamingTariff(ServiceTariff):
16     def calculate(self, units):
17         return units * 5.0
18
19 def process_billing(tariff, amount):
20     print(f"Тариф {tariff.__class__.__name__}: до сплати {tariff.calculate(amount)} грн")
21
22 voice = VoiceTariff()
23 data = DataTariff()
24 roaming = RoamingTariff()
25
26 process_billing(voice, amount=100)
```

Run console output:

```
Тариф VoiceTariff: до сплати 75.0 грн
Тариф DataTariff: до сплати 150.0 грн
Тариф RoamingTariff: до сплати 500.0 грн
```

Рис. 3(Завдання 2.1-2.2)

```
1 #3.1
2 class NetworkConnection():
3     def connect(self):
4         print("Connecting to the Network...")
5
6 class WifiConnection(NetworkConnection):
7     def connect(self):
8         print("WiFi connected!")
9
10 class LteConnection(NetworkConnection):
11     def connect(self):
12         print("LTE connected!")
13
14 def establish_communication(connection: NetworkConnection):
15     connection.connect()
16
17 wifi = WifiConnection()
18 lte = LteConnection()
19
20 establish_communication(wifi)
21 establish_communication(lte)
```

Run console output:

```
C:\Users\Maksym\Desktop\python\python.exe "C:\Users\Maksym\Desktop\PYTHON\OOP2_0\lab1(3.1).py"
WiFi connected!
LTE connected!
Process finished with exit code 0
```

Рис.4(Завдання 3.1)

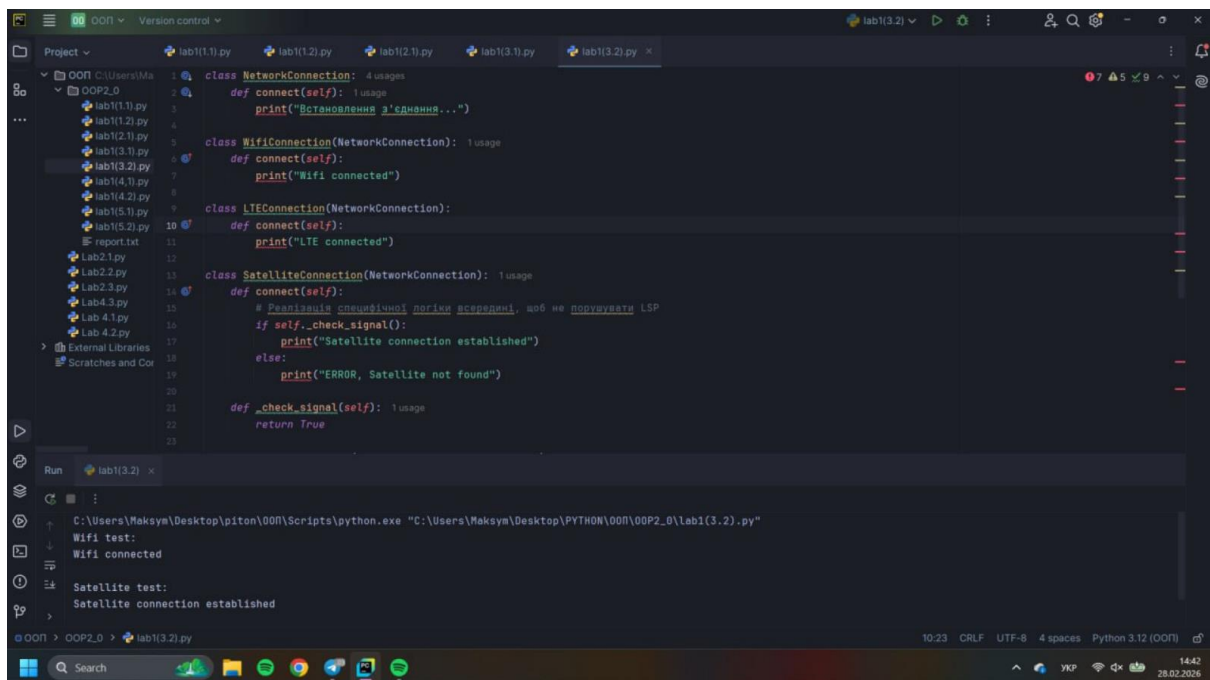


Рис.5(Завдання 3.2)

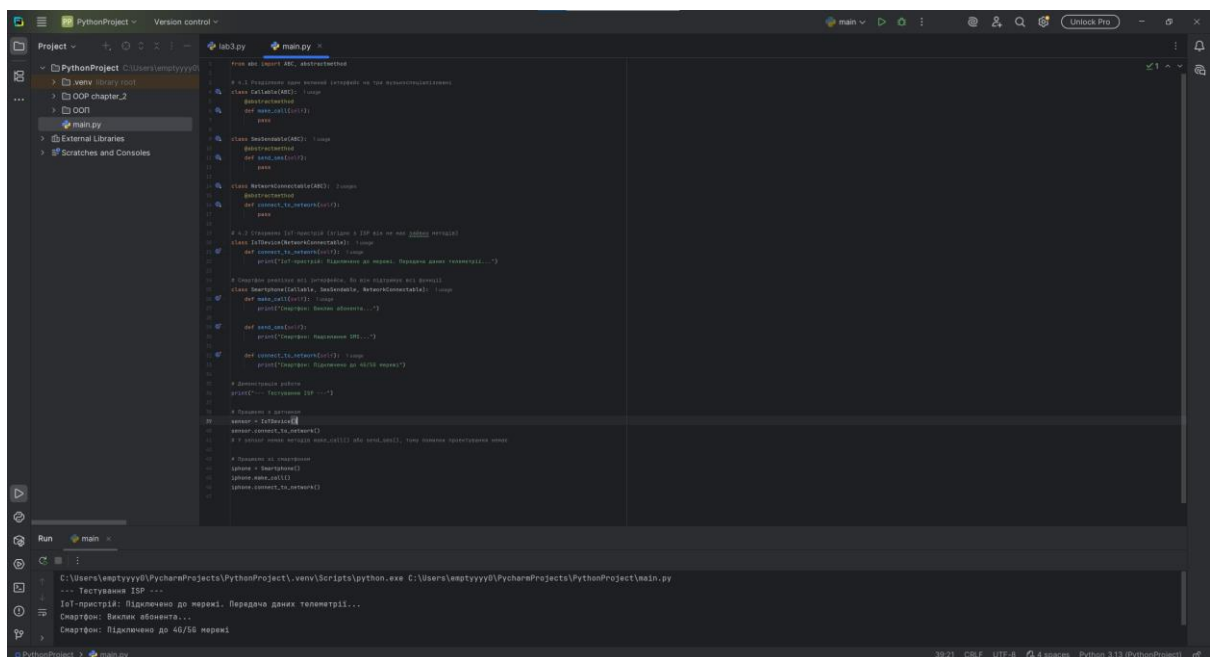


Рис.6(Завдання 4.1)

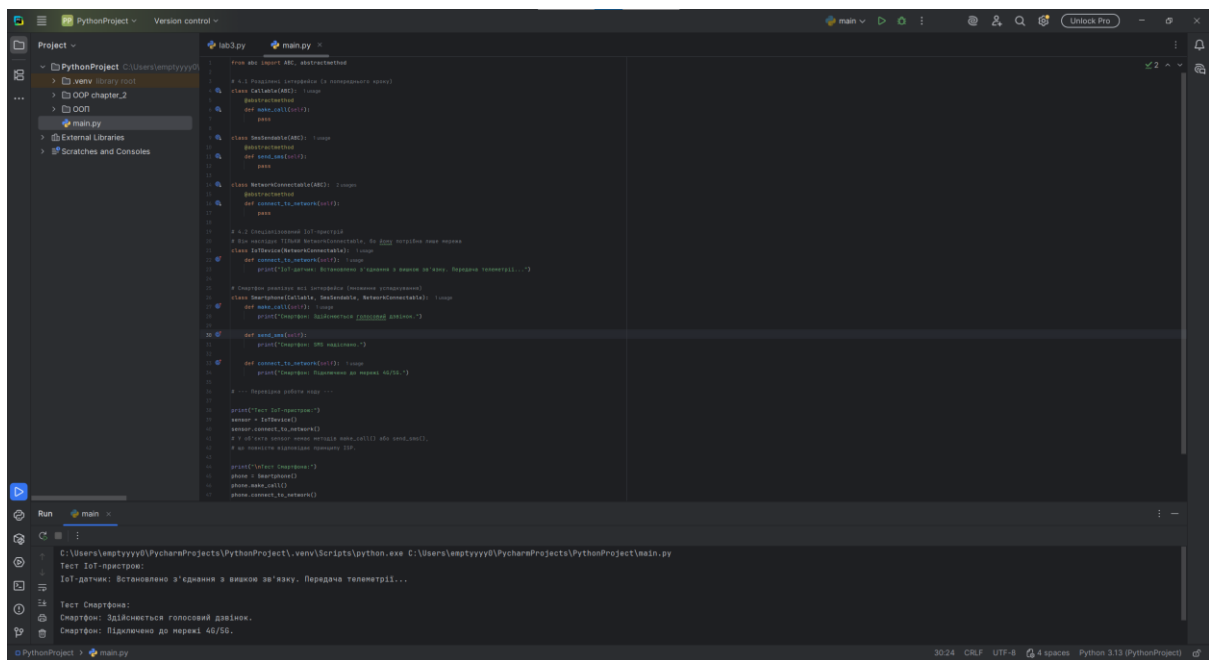


Рис.7(Завдання 4.2)

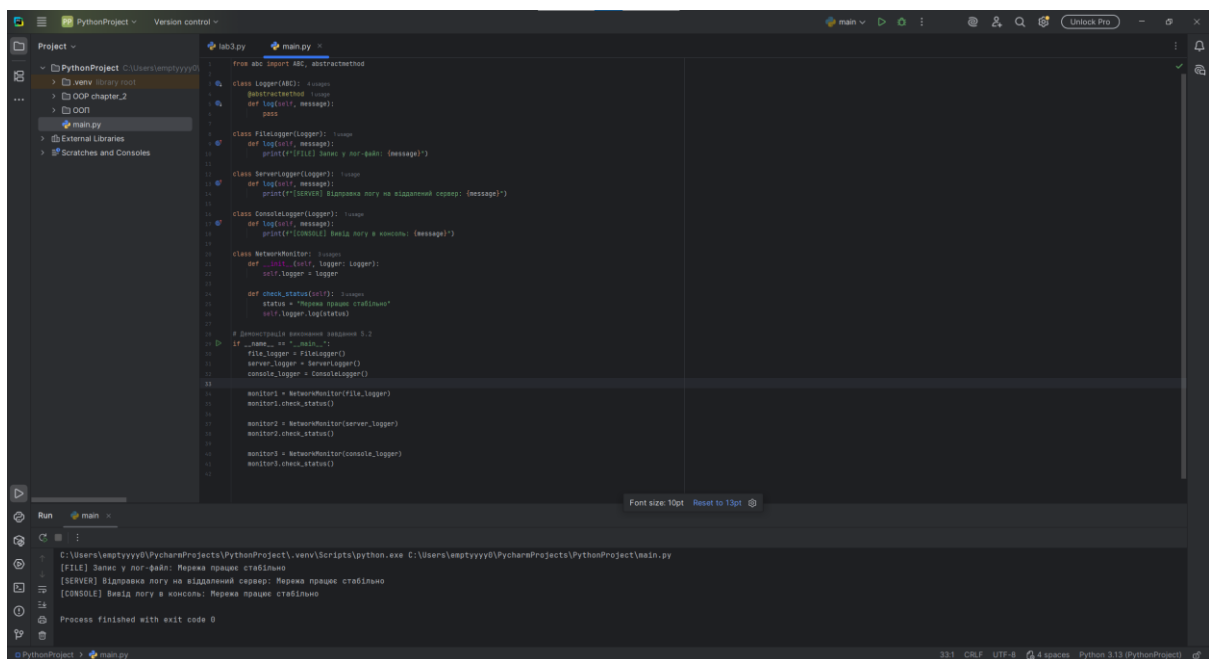


Рис.8(Завдання 5.1)

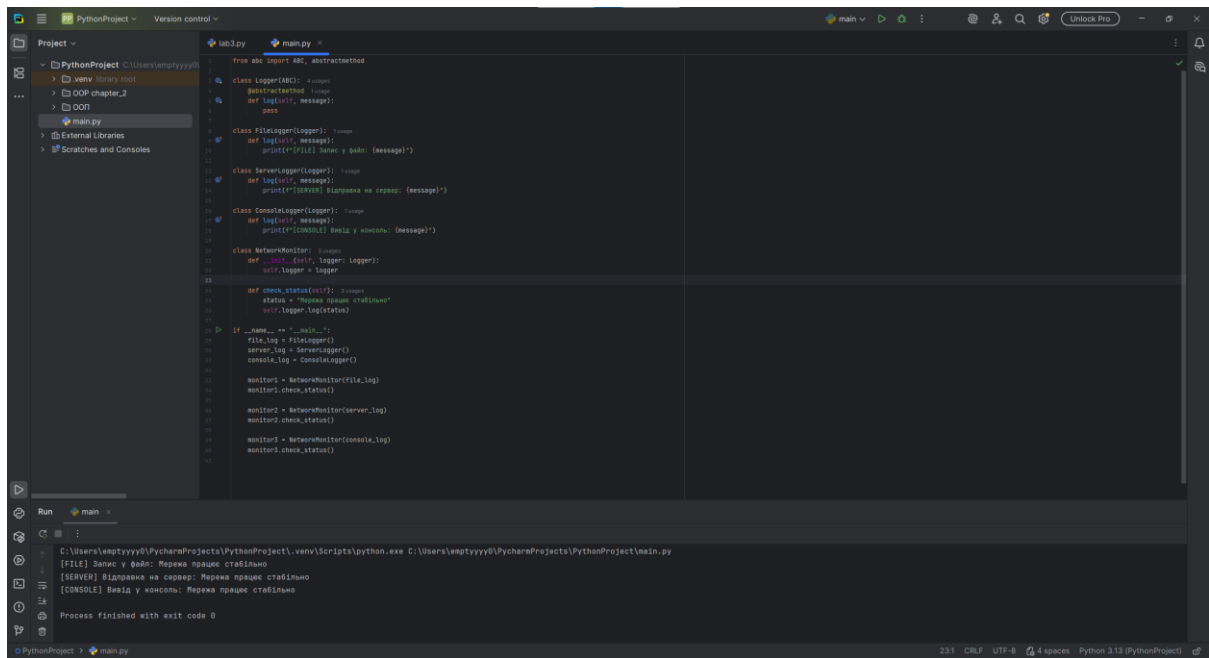


Рис.9(Завдання 5.2)

Висновок

У ході виконання роботи я опанував п'ять принципів **SOLID** для проєктування систем у телекомунікаціях:

- **SRP:** Навчився розділяти класи за функціями (звітність, розрахунок, збереження), щоб кожна сутність мала лише одну відповідальність.
- **OCP:** Реалізував можливість додавання нових тарифів без зміни основного коду.
- **LSP:** Забезпечив взаємозамінність типів підключення (LTE, WiFi) без порушення логіки програми.
- **ISP:** Розділив громіздкі інтерфейси на менші, щоб пристрої не залежали від непотрібних їм методів (наприклад, SMS для IoT).
- **DIP:** Перепроєктував систему логування так, щоб високорівневі модулі залежали від абстракцій, а не від конкретних реалізацій.

Застосування цих принципів дозволяє створювати гнучкий та масштабований код, який легко тестувати та підтримувати у реальних проєктах

